SAP-8

Simple Accumulator Processor
An 8-bit CPU: Design, Simulation and Programs in Assembly and C

Team Members

Name	Part	Contribution
Sreya	Part A	SAP-8 design: ISA, assembler, simulator (sap8.py), block diagram, this report
Lavya	Part B	Program 1: 5! written directly in SAP-8 assembly (prog1_factorial.py)
Manasa	Part C	Program 2: 2^4 in C (prog2_power.c) and its translation (prog2_power.py)

CMPE-220 | CPU Design Project
November 2026

Program	Written in	Result
Factorial 5!	SAP-8 assembly (directly)	120
Exponent 2^4	C, translated by hand to assembly	16

1. CPU Design

Table 1. *SAP-8 specification*

Property	SAP-8
Architecture	Accumulator machine; code and data share one RAM
Data path	8 bits; arithmetic wraps modulo 256
RAM	256 bytes (0x00 - 0xFF); code segment 0x00 - 0x3F, data segment 0x40 - 0xFF
Registers	A (accumulator), B (temp), PC (program counter); internal IR and OPR
Flag	Z (zero)
Instruction format	1 opcode byte, plus 1 operand byte for 2-byte instructions
Instruction set	15 instructions (see Table 3)
Cycle	Fetch -> Decode -> Execute (simulator prints RAM after every cycle)

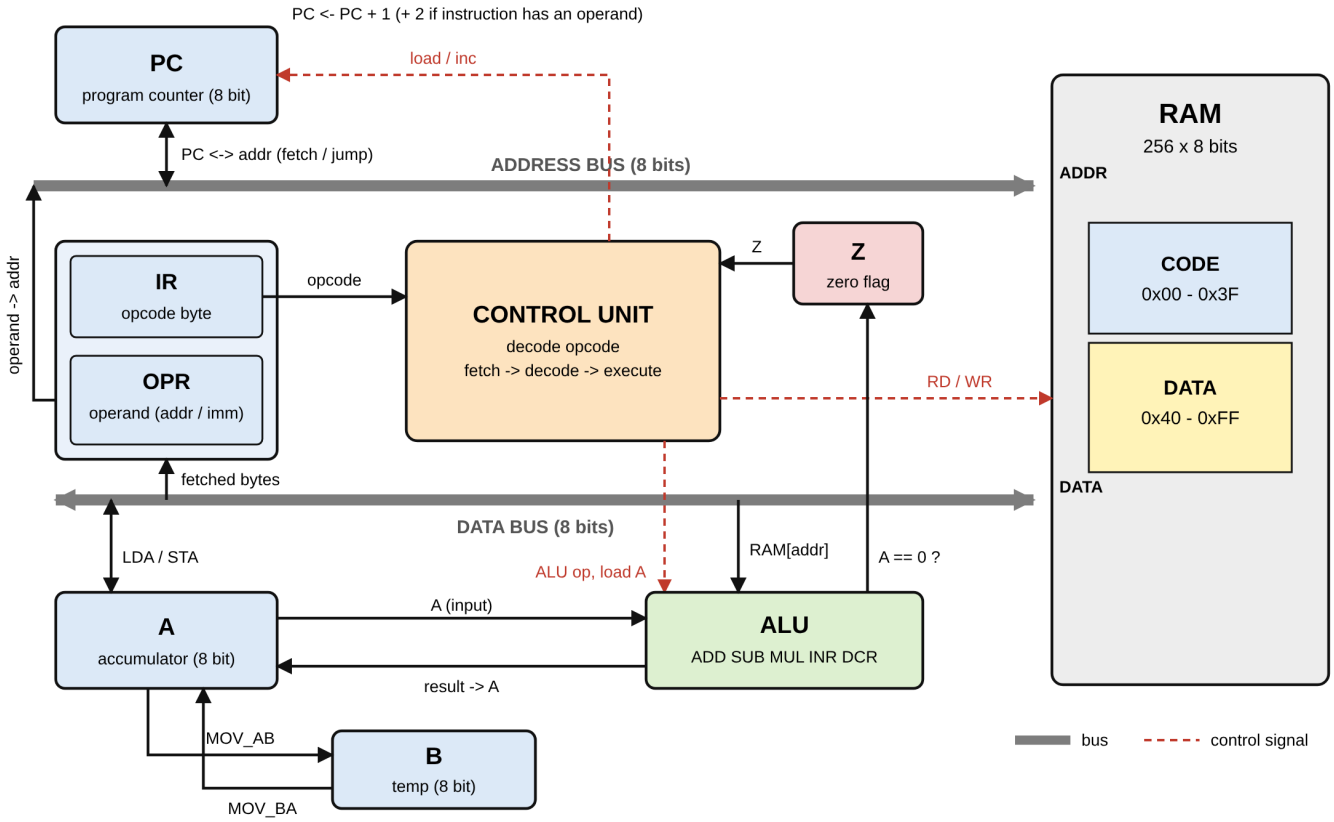


Figure 1. *SAP-8 block diagram (registers, ALU, control unit, buses, RAM)*

Table 2. *Fetch-decode-execute steps*

#	Phase	What happens
1	Fetch	IR <- RAM[PC] (opcode byte)
2	Fetch	If the instruction has an operand: OPR <- RAM[PC+1]
3	Advance	PC <- PC + 1 (1-byte) or PC + 2 (2-byte)
4	Decode	Control unit reads the opcode in IR
5	Execute	Memory op, ALU op (updates Z), register move, or jump (tests Z, may load PC)

2. Instruction Encoding

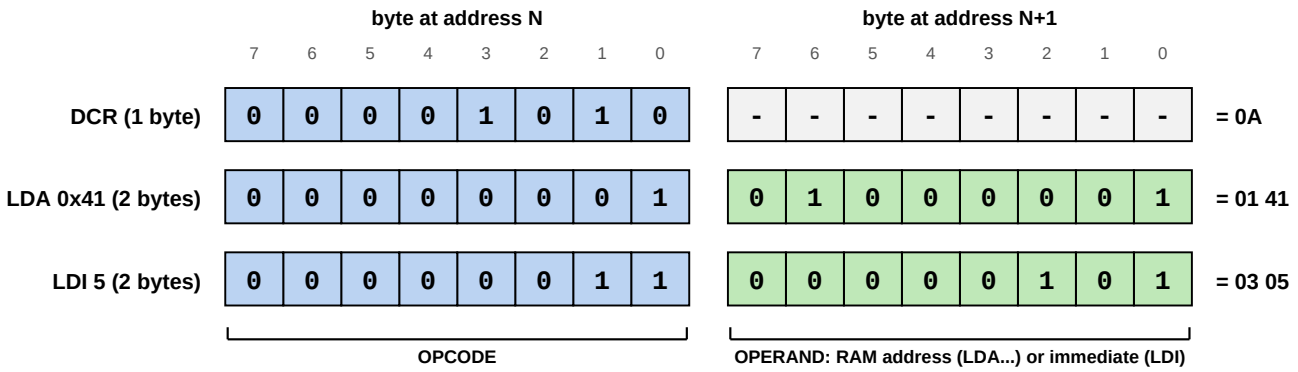


Figure 2. *Instruction formats: 1-byte and 2-byte, with worked examples*

Table 3. *SAP-8 instruction set and encoding*

Opcode	Binary	Mnemonic	Operand	Bytes	Operation	Z flag	Example = bytes
00	00000000	NOP	--	1	no operation	unchanged	NOP = 00
01	00000001	LDA	addr	2	A <- RAM[addr]	set	LDA 0x41 = 01 41
02	00000010	STA	addr	2	RAM[addr] <- A	unchanged	STA 0x41 = 02 41
03	00000011	LDI	imm	2	A <- immediate	set	LDI 5 = 03 05
04	00000100	ADD	addr	2	A <- A + RAM[addr]	set	ADD 0x40 = 04 40
05	00000101	SUB	addr	2	A <- A - RAM[addr]	set	SUB 0x40 = 05 40
06	00000110	MUL	addr	2	A <- A * RAM[addr]	set	MUL 0x40 = 06 40
07	00000111	JMP	addr	2	PC <- addr	unchanged	JMP 0x08 = 07 08
08	00001000	JZ	addr	2	if Z==1: PC <- addr	unchanged	JZ 0x11 = 08 11
09	00001001	JNZ	addr	2	if Z==0: PC <- addr	unchanged	JNZ 0x08 = 09 08
0A	00001010	DCR	--	1	A <- A - 1	set	DCR = 0A
0B	00001011	INR	--	1	A <- A + 1	set	INR = 0B
0C	00001100	MOV_AB	--	1	B <- A	unchanged	MOV_AB = 0C
0D	00001101	MOV_BA	--	1	A <- B	set	MOV_BA = 0D
FF	11111111	HLT	--	1	halt	unchanged	HLT = FF

Table 4. *Z flag rule: Z = 1 when the last result written to A was 0*

Event	A after	Z
LDI 0	00	1
DCR on A = 01	00	1
INR on A = FF (8-bit wrap)	00	1
DCR on A = 05	04	0
STA / MOV_AB / jumps	unchanged	unchanged

3. Program 1: Factorial 5! (written directly in assembly)

Table 5. *Program 1 source, assembled (blue = code, yellow = data)*

Addr	Bytes	Label	Instruction	Comment
0x00	03 05		LDI 5	A = 5
0x02	02 40		STA N	N = 5 (loop counter)
0x04	03 01		LDI 1	A = 1
0x06	02 41		STA RES	RES = 1 (running product)
0x08	01 41	loop	LDA RES	A = RES
0x0A	06 40		MUL N	A = RES * N
0x0C	02 41		STA RES	RES = A
0x0E	01 40		LDA N	A = N
0x10	0A		DCR	A = N - 1 (sets the Z flag)
0x11	02 40		STA N	N = N - 1 (STA does not change Z)
0x13	09 08		JNZ loop	repeat while N != 0
0x15	FF		HLT	done, answer is in RES
0x40	00	N	.byte 0	loop counter
0x41	00	RES	.byte 0	result (5! = 120 = 0x78)

Table 6. *Program 1 initial RAM (before step 1); only 16-byte rows that hold code or data are shown*

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	03	05	02	40	03	01	02	41	01	41	06	40	02	41	01	40
10:	0A	02	40	09	08	FF	00	00	00	00	00	00	00	00	00	00
40:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

code data

Table 7. *Program 1 execution trace: RAM after each instruction (RAM = hex, vars = decimal)*

instruction just executed (opcode + operand) byte written next instruction (PC)

#01 0x00: **LDI 5**
A 00->05 B=00 Z=0 PC->0x02
vars: N=0 RES=0

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	03	05	02	40	03	01	02	41	01	41	06	40	02	41	01	40
10:	0A	02	40	09	08	FF	00	00	00	00	00	00	00	00	00	00
40:	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

#02 0x02: **STA N**
A 05->05 B=00 Z=0 PC->0x04
vars: N=5 RES=0

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	03	05	02	40	03	01	02	41	01	41	06	40	02	41	01	40
10:	0A	02	40	09	08	FF	00	00	00	00	00	00	00	00	00	00
40:	05	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

#03 0x04: **LDI 1**
A 05->01 B=00 Z=0 PC->0x06
vars: N=5 RES=0

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	03	05	02	40	03	01	02	41	01	41	06	40	02	41	01	40
10:	0A	02	40	09	08	FF	00	00	00	00	00	00	00	00	00	00
40:	05	00	00	00	00	00	00	00	00	00	00	00	00	00	00	00

#04 0x06: **STA RES**
A 01->01 B=00 Z=0 PC->0x08
vars: N=5 RES=1

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	03	05	02	40	03	01	02	41	01	41	06	40	02	41	01	40
10:	0A	02	40	09	08	FF	00	00	00	00	00	00	00	00	00	00
40:	05	01	00	00	00	00	00	00	00	00	00	00	00	00	00	00

#05 0x08: **LDA RES**
A 01->01 B=00 Z=0 PC->0x0A
vars: N=5 RES=1

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	03	05	02	40	03	01	02	41	01	41	06	40	02	41	01	40
10:	0A	02	40	09	08	FF	00	00	00	00	00	00	00	00	00	00
40:	05	01	00	00	00	00	00	00	00	00	00	00	00	00	00	00

#06 0x0A: **MUL N**
A 01->05 B=00 Z=0 PC->0x0C
vars: N=5 RES=1

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	03	05	02	40	03	01	02	41	01	41	06	40	02	41	01	40
10:	0A	02	40	09	08	FF	00	00	00	00	00	00	00	00	00	00
40:	05	01	00	00	00	00	00	00	00	00	00	00	00	00	00	00

#07 0x0C: **STA RES**
A 05->05 B=00 Z=0 PC->0x0E
vars: N=5 RES=5

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	03	05	02	40	03	01	02	41	01	41	06	40	02	41	01	40
10:	0A	02	40	09	08	FF	00	00	00	00	00	00	00	00	00	00
40:	05	05	00	00	00	00	00	00	00	00	00	00	00	00	00	00

#08 0x0E: **LDA N**
A 05->05 B=00 Z=0 PC->0x10
vars: N=5 RES=5

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	03	05	02	40	03	01	02	41	01	41	06	40	02	41	01	40
10:	0A	02	40	09	08	FF	00	00	00	00	00	00	00	00	00	00
40:	05	05	00	00	00	00	00	00	00	00	00	00	00	00	00	00

#09 0x10: **DCR**
A 05->04 B=00 Z=0 PC->0x11
vars: N=5 RES=5

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	03	05	02	40	03	01	02	41	01	41	06	40	02	41	01	40
10:	0A	02	40	09	08	FF	00	00	00	00	00	00	00	00	00	00
40:	05	05	00	00	00	00	00	00	00	00	00	00	00	00	00	00

#10 0x11: **STA N**
A 04->04 B=00 Z=0 PC->0x13
vars: N=4 RES=5

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	03	05	02	40	03	01	02	41	01	41	06	40	02	41	01	40
10:	0A	02	40	09	08	FF	00	00	00	00	00	00	00	00	00	00
40:	04	05	00	00	00	00	00	00	00	00	00	00	00	00	00	00

#11 0x13: **JNZ loop**
A 04->04 B=00 Z=0 PC->0x08
vars: N=4 RES=5

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	03	05	02	40	03	01	02	41	01	41	06	40	02	41	01	40
10:	0A	02	40	09	08	FF	00	00	00	00	00	00	00	00	00	00
40:	04	05	00	00	00	00	00	00	00	00	00	00	00	00	00	00

#12 0x08: **LDA RES**
A 04->05 B=00 Z=0 PC->0x0A
vars: N=4 RES=5

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	03	05	02	40	03	01	02	41	01	41	06	40	02	41	01	40
10:	0A	02	40	09	08	FF	00	00	00	00	00	00	00	00	00	00
40:	04	05	00	00	00	00	00	00	00	00	00	00	00	00	00	00

#13 0x0A: **MUL N**
A 05->14 B=00 Z=0 PC->0x0C
vars: N=4 RES=5

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	03	05	02	40	03	01	02	41	01	41	06	40	02	41	01	40
10:	0A	02	40	09	08	FF	00	00	00	00	00	00	00	00	00	00
40:	04	05	00	00	00	00	00	00	00	00	00	00	00	00	00	00

#14 0x0C: **STA RES**
A 14->14 B=00 Z=0 PC->0x0E
vars: N=4 RES=20

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	03	05	02	40	03	01	02	41	01	41	06	40	02	41	01	40
10:	0A	02	40	09	08	FF	00	00	00	00	00	00	00	00	00	00
40:	04	14	00	00	00	00	00	00	00	00	00	00	00	00	00	00

#15 0x0E: **LDA N**
A 14->04 B=00 Z=0 PC->0x10
vars: N=4 RES=20

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	03	05	02	40	03	01	02	41	01	41	06	40	02	41	01	40
10:	0A	02	40	09	08	FF	00	00	00	00	00	00	00	00	00	00
40:	04	14	00	00	00	00	00	00	00	00	00	00	00	00	00	00

#16 0x10: **DCR**
A 04->03 B=00 Z=0 PC->0x11
vars: N=4 RES=20

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	03	05	02	40	03	01	02	41	01	41	06	40	02	41	01	40
10:	0A	02	40	09	08	FF	00	00	00	00	00	00	00	00	00	00
40:	04	14	00	00	00	00	00	00	00	00	00	00	00	00	00	00

#17 0x11: **STA N**
A 03->03 B=00 Z=0 PC->0x13
vars: N=3 RES=20

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	03	05	02	40	03	01	02	41	01	41	06	40	02	41	01	40
10:	0A	02	40	09	08	FF	00	00	00	00	00	00	00	00	00	00
40:	03	14	00	00	00	00	00	00	00	00	00	00	00	00	00	00

#18 0x13: **JNZ loop**
A 03->03 B=00 Z=0 PC->0x08
vars: N=3 RES=20

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	03	05	02	40	03	01	02	41	01	41	06	40	02	41	01	40
10:	0A	02	40	09	08	FF	00	00	00	00	00	00	00	00	00	00
40:	03	14	00	00	00	00	00	00	00	00	00	00	00	00	00	00

#19 0x08: **LDA RES**
A 03->14 B=00 Z=0 PC->0x0A
vars: N=3 RES=20

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	03	05	02	40	03	01	02	41	01	41	06	40	02	41	01	40
10:	0A	02	40	09	08	FF	00	00	00	00	00	00	00	00	00	00
40:	03	14	00	00	00	00	00	00	00	00	00	00	00	00	00	00

#20 0x0A: **MUL N**
A 14->3C B=00 Z=0 PC->0x0C
vars: N=3 RES=20

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	03	05	02	40	03	01	02	41	01	41	06	40	02	41	01	40
10:	0A	02	40	09	08	FF	00	00	00	00	00	00	00	00	00	00
40:	03	14	00	00	00	00	00	00	00	00	00	00	00	00	00	00

#21 0x0C: **STA RES**
A 3C->3C B=00 Z=0 PC->0x0E
vars: N=3 RES=60

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	03	05	02	40	03	01	02	41	01	41	06	40	02	41	01	40
10:	0A	02	40	09	08	FF	00	00	00	00	00	00	00	00	00	00
40:	03	3C	00	00	00	00	00	00	00	00	00	00	00	00	00	00

#22 0x0E: **LDA N**
A 3C->03 B=00 Z=0 PC->0x10
vars: N=3 RES=60

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	03	05	02	40	03	01	02	41	01	41	06	40	02	41	01	40
10:	0A	02	40	09	08	FF	00	00	00	00	00	00	00	00	00	00
40:	03	3C	00	00	00	00	00	00	00	00	00	00	00	00	00	00

#23 0x10: **DCR**
A 03->02 B=00 Z=0 PC->0x11
vars: N=3 RES=60

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	03	05	02	40	03	01	02	41	01	41	06	40	02	41	01	40
10:	0A	02	40	09	08	FF	00	00	00	00	00	00	00	00	00	00
40:	03	3C	00	00	00	00	00	00	00	00	00	00	00	00	00	00

#24 0x11: **STA N**
A 02->02 B=00 Z=0 PC->0x13
vars: N=2 RES=60

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	03	05	02	40	03	01	02	41	01	41	06	40	02	41	01	40
10:	0A	02	40	09	08	FF	00	00	00	00	00	00	00	00	00	00
40:	02	3C	00	00	00	00	00	00	00	00	00	00	00	00	00	00

#25 0x13: **JNZ loop**
A 02->02 B=00 Z=0 PC->0x08
vars: N=2 RES=60

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	03	05	02	40	03	01	02	41	01	41	06	40	02	41	01	40
10:	0A	02	40	09	08	FF	00	00	00	00	00	00	00	00	00	00
40:	02	3C	00	00	00	00	00	00	00	00	00	00	00	00	00	00

#26 0x08: **LDA RES**
A 02->3C B=00 Z=0 PC->0x0A
vars: N=2 RES=60

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	03	05	02	40	03	01	02	41	01	41	06	40	02	41	01	40
10:	0A	02	40	09	08	FF	00	00	00	00	00	00	00	00	00	00
40:	02	3C	00	00	00	00	00	00	00	00	00	00	00	00	00	00

#27 0x0A: **MUL N**
A 3C->78 B=00 Z=0 PC->0x0C
vars: N=2 RES=60

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	03	05	02	40	03	01	02	41	01	41	06	40	02	41	01	40
10:	0A	02	40	09	08	FF	00	00	00	00	00	00	00	00	00	00
40:	02	3C	00	00	00	00	00	00	00	00	00	00	00	00	00	00

#28 0x0C: **STA RES**
A 78->78 B=00 Z=0 PC->0x0E
vars: N=2 RES=120

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	03	05	02	40	03	01	02	41	01	41	06	40	02	41	01	40
10:	0A	02	40	09	08	FF	00	00	00	00	00	00	00	00	00	00
40:	02	78	00	00	00	00	00	00	00	00	00	00	00	00	00	00

#29 0x0E: **LDA N**
A 78->02 B=00 Z=0 PC->0x10
vars: N=2 RES=120

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	03	05	02	40	03	01	02	41	01	41	06	40	02	41	01	40
10:	0A	02	40	09	08	FF	00	00	00	00	00	00	00	00	00	00
40:	02	78	00	00	00	00	00	00	00	00	00	00	00	00	00	00

#30 0x10: **DCR**
A 02->01 B=00 Z=0 PC->0x11
vars: N=2 RES=120

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	03	05	02	40	03	01	02	41	01	41	06	40	02	41	01	40
10:	0A	02	40	09	08	FF	00	00	00	00	00	00	00	00	00	00
40:	02	78	00	00	00	00	00	00	00	00	00	00	00	00	00	00

#31 0x11: **STA N**
A 01->01 B=00 Z=0 PC->0x13
vars: N=1 RES=120

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	03	05	02	40	03	01	02	41	01	41	06	40	02	41	01	40
10:	0A	02	40	09	08	FF	00	00	00	00	00	00	00	00	00	00
40:	01	78	00	00	00	00	00	00	00	00	00	00	00	00	00	00

#32 0x13: **JNZ loop**
A 01->01 B=00 Z=0 PC->0x08
vars: N=1 RES=120

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	03	05	02	40	03	01	02	41	01	41	06	40	02	41	01	40
10:	0A	02	40	09	08	FF	00	00	00	00	00	00	00	00	00	00
40:	01	78	00	00	00	00	00	00	00	00	00	00	00	00	00	00

#33 0x08: **LDA RES**
A 01->78 B=00 Z=0 PC->0x0A
vars: N=1 RES=120

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	03	05	02	40	03	01	02	41	01	41	06	40	02	41	01	40
10:	0A	02	40	09	08	FF	00	00	00	00	00	00	00	00	00	00
40:	01	78	00	00	00	00	00	00	00	00	00	00	00	00	00	00

#34 0x0A: **MUL N**
A 78->78 B=00 Z=0 PC->0x0C
vars: N=1 RES=120

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	03	05	02	40	03	01	02	41	01	41	06	40	02	41	01	40
10:	0A	02	40	09	08	FF	00	00	00	00	00	00	00	00	00	00
40:	01	78	00	00	00	00	00	00	00	00	00	00	00	00	00	00

#35 0x0C: **STA RES**
A 78->78 B=00 Z=0 PC->0x0E
vars: N=1 RES=120

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	03	05	02	40	03	01	02	41	01	41	06	40	02	41	01	40
10:	0A	02	40	09	08	FF	00	00	00	00	00	00	00	00	00	00
40:	01	78	00	00	00	00	00	00	00	00	00	00	00	00	00	00

#36 0x0E: **LDA N**
A 78->01 B=00 Z=0 PC->0x10
vars: N=1 RES=120

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	03	05	02	40	03	01	02	41	01	41	06	40	02	41	01	40
10:	0A	02	40	09	08	FF	00	00	00	00	00	00	00	00	00	00
40:	01	78	00	00	00	00	00	00	00	00	00	00	00	00	00	00

#37 0x10: **DCR**
A 01->00 B=00 Z=1 PC->0x11
vars: N=1 RES=120

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	03	05	02	40	03	01	02	41	01	41	06	40	02	41	01	40
10:	0A	02	40	09	08	FF	00	00	00	00	00	00	00	00	00	00
40:	01	78	00	00	00	00	00	00	00	00	00	00	00	00	00	00

#38 0x11: **STA N**
A 00->00 B=00 Z=1 PC->0x13
vars: N=0 RES=120

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	03	05	02	40	03	01	02	41	01	41	06	40	02	41	01	40
10:	0A	02	40	09	08	FF	00	00	00	00	00	00	00	00	00	00
40:	00	78	00	00	00	00	00	00	00	00	00	00	00	00	00	00

#39 0x13: **JNZ loop**
A 00->00 B=00 Z=1 PC->0x15
vars: N=0 RES=120

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	03	05	02	40	03	01	02	41	01	41	06	40	02	41	01	40
10:	0A	02	40	09	08	FF	00	00	00	00	00	00	00	00	00	00
40:	00	78	00	00	00	00	00	00	00	00	00	00	00	00	00	00

#40 0x15: **HLT**
A 00->00 B=00 Z=1 PC->0x16
vars: N=0 RES=120

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	03	05	02	40	03	01	02	41	01	41	06	40	02	41	01	40
10:	0A	02	40	09	08	FF	00	00	00	00	00	00	00	00	00	00
40:	00	78	00	00	00	00	00	00	00	00	00	00	00	00	00	00

Table 8. *Program 1 result*

Result variable	CPU result	Expected (Python)	Instructions executed	Verdict
RES @ 0x41	**120** (0x78)	math.factorial(5) = 120	40	**PASS**

4. Program 2: Exponent 2^4 (written in C, translated to assembly)

Table 9. *Program 2 C source (prog2_power.c)*

```
 1  /* prog2_power.c -- PROGRAM 2 source (Part C, Manasa)
 2   * Computes 2^4 = 16 by repeated multiplication.
 3   * Hand-translated to SAP-8 assembly in prog2_power.py.
 4   *
 5   * Host check:  gcc -Wall prog2_power.c -o power && ./power     (prints 16)
 6   */
 7  #include <stdio.h>
 8
 9  int base   = 2;                 /* initialised globals -> data segment 0x40.. */
10  int expo   = 4;
11  int result = 1;
12
13  int main(void) {
14      while (expo != 0) {         /* loop: LDA expo / JZ end                    */
15          result = result * base;  /* LDA result / MUL base / STA result        */
16          expo   = expo - 1;       /* LDA expo / DCR / STA expo                 */
17      }                           /* JMP loop                                   */
18      printf("%d\n", result);     /* host check only (not translated)           */
19      return 0;                   /* end: HLT                                   */
20  }
```

Table 10. *C-to-assembly translation (line by line)*

C statement	SAP-8 assembly	Bytes

`int base=2, expo=4, result=1;`	`.byte 2`
	`.byte 4`
	`.byte 1`
		`02`
		`04`
		`01`

`while (expo != 0)`	`LDA expo` `JZ end`	`01 41` `08 11`

`  result = result * base;`	`LDA result`
	`MUL base`
	`STA result`
		`01 42`
		`06 40`
		`02 42`

`  expo = expo - 1;`	`LDA expo`
	`DCR`
	`STA expo`
		`01 41`
		`0A`
		`02 41`

`}   /* end of loop body */`	`JMP loop`	`07 00`
`return 0;`	`HLT`	`FF`

Table 11. *Program 2 source, assembled (blue = code, yellow = data)*

Addr	Bytes	Label	Instruction	Comment
0x00	01 41	loop	LDA expo	while (expo != 0) -- load expo, sets Z
0x02	08 11		JZ end	leave the loop when expo == 0
0x04	01 42		LDA result	result = result * base
0x06	06 40		MUL base	
0x08	02 42		STA result	
0x0A	01 41		LDA expo	expo = expo - 1
0x0C	0A		DCR	
0x0D	02 41		STA expo	
0x0F	07 00		JMP loop	back to the while test
0x11	FF	end	HLT	return 0
0x40	02	base	.byte 2	int base = 2;
0x41	04	expo	.byte 4	int expo = 4;
0x42	01	result	.byte 1	int result = 1; (final value 16 = 0x10)

Table 12. *Program 2 initial RAM (before step 1); only 16-byte rows that hold code or data are shown*

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	01	41	08	11	01	42	06	40	02	42	01	41	0A	02	41	07
10:	00	FF	00	00	00	00	00	00	00	00	00	00	00	00	00	00
40:	02	04	01	00	00	00	00	00	00	00	00	00	00	00	00	00

code data

Table 13. *Program 2 execution trace: RAM after each instruction (RAM = hex, vars = decimal)*

instruction just executed (opcode + operand) byte written next instruction (PC)

#01 0x00: **LDA expo**
A 00->04 B=00 Z=0 PC->0x02
vars: expo=4 result=1

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	01	41	08	11	01	42	06	40	02	42	01	41	0A	02	41	07
10:	00	FF	00	00	00	00	00	00	00	00	00	00	00	00	00	00
40:	02	04	01	00	00	00	00	00	00	00	00	00	00	00	00	00

#02 0x02: **JZ end**
A 04->04 B=00 Z=0 PC->0x04
vars: expo=4 result=1

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	01	41	08	11	01	42	06	40	02	42	01	41	0A	02	41	07
10:	00	FF	00	00	00	00	00	00	00	00	00	00	00	00	00	00
40:	02	04	01	00	00	00	00	00	00	00	00	00	00	00	00	00

#03 0x04: **LDA result**
A 04->01 B=00 Z=0 PC->0x06
vars: expo=4 result=1

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	01	41	08	11	01	42	06	40	02	42	01	41	0A	02	41	07
10:	00	FF	00	00	00	00	00	00	00	00	00	00	00	00	00	00
40:	02	04	01	00	00	00	00	00	00	00	00	00	00	00	00	00

#04 0x06: **MUL base**
A 01->02 B=00 Z=0 PC->0x08
vars: expo=4 result=1

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	01	41	08	11	01	42	06	40	02	42	01	41	0A	02	41	07
10:	00	FF	00	00	00	00	00	00	00	00	00	00	00	00	00	00
40:	02	04	01	00	00	00	00	00	00	00	00	00	00	00	00	00

#05 0x08: **STA result**
A 02->02 B=00 Z=0 PC->0x0A
vars: expo=4 result=2

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	01	41	08	11	01	42	06	40	02	42	01	41	0A	02	41	07
10:	00	FF	00	00	00	00	00	00	00	00	00	00	00	00	00	00
40:	02	04	02	00	00	00	00	00	00	00	00	00	00	00	00	00

#06 0x0A: **LDA expo**
A 02->04 B=00 Z=0 PC->0x0C
vars: expo=4 result=2

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	01	41	08	11	01	42	06	40	02	42	01	41	0A	02	41	07
10:	00	FF	00	00	00	00	00	00	00	00	00	00	00	00	00	00
40:	02	04	02	00	00	00	00	00	00	00	00	00	00	00	00	00

#07 0x0C: **DCR**
A 04->03 B=00 Z=0 PC->0x0D
vars: expo=4 result=2

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	01	41	08	11	01	42	06	40	02	42	01	41	0A	02	41	07
10:	00	FF	00	00	00	00	00	00	00	00	00	00	00	00	00	00
40:	02	04	02	00	00	00	00	00	00	00	00	00	00	00	00	00

#08 0x0D: **STA expo**
A 03->03 B=00 Z=0 PC->0x0F
vars: expo=3 result=2

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	01	41	08	11	01	42	06	40	02	42	01	41	0A	02	41	07
10:	00	FF	00	00	00	00	00	00	00	00	00	00	00	00	00	00
40:	02	03	02	00	00	00	00	00	00	00	00	00	00	00	00	00

#09 0x0F: **JMP loop**
A 03->03 B=00 Z=0 PC->0x00
vars: expo=3 result=2

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	01	41	08	11	01	42	06	40	02	42	01	41	0A	02	41	07
10:	00	FF	00	00	00	00	00	00	00	00	00	00	00	00	00	00
40:	02	03	02	00	00	00	00	00	00	00	00	00	00	00	00	00

#10 0x00: **LDA expo**
A 03->03 B=00 Z=0 PC->0x02
vars: expo=3 result=2

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	01	41	08	11	01	42	06	40	02	42	01	41	0A	02	41	07
10:	00	FF	00	00	00	00	00	00	00	00	00	00	00	00	00	00
40:	02	03	02	00	00	00	00	00	00	00	00	00	00	00	00	00

#11 0x02: **JZ end**
A 03->03 B=00 Z=0 PC->0x04
vars: expo=3 result=2

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	01	41	08	11	01	42	06	40	02	42	01	41	0A	02	41	07
10:	00	FF	00	00	00	00	00	00	00	00	00	00	00	00	00	00
40:	02	03	02	00	00	00	00	00	00	00	00	00	00	00	00	00

#12 0x04: **LDA result**
A 03->02 B=00 Z=0 PC->0x06
vars: expo=3 result=2

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	01	41	08	11	01	42	06	40	02	42	01	41	0A	02	41	07
10:	00	FF	00	00	00	00	00	00	00	00	00	00	00	00	00	00
40:	02	03	02	00	00	00	00	00	00	00	00	00	00	00	00	00

#13 0x06: **MUL base**
A 02->04 B=00 Z=0 PC->0x08
vars: expo=3 result=2

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	01	41	08	11	01	42	06	40	02	42	01	41	0A	02	41	07
10:	00	FF	00	00	00	00	00	00	00	00	00	00	00	00	00	00
40:	02	03	02	00	00	00	00	00	00	00	00	00	00	00	00	00

#14 0x08: **STA result**
A 04->04 B=00 Z=0 PC->0x0A
vars: expo=3 result=4

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	01	41	08	11	01	42	06	40	02	42	01	41	0A	02	41	07
10:	00	FF	00	00	00	00	00	00	00	00	00	00	00	00	00	00
40:	02	03	04	00	00	00	00	00	00	00	00	00	00	00	00	00

#15 0x0A: **LDA expo**
A 04->03 B=00 Z=0 PC->0x0C
vars: expo=3 result=4

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	01	41	08	11	01	42	06	40	02	42	01	41	0A	02	41	07
10:	00	FF	00	00	00	00	00	00	00	00	00	00	00	00	00	00
40:	02	03	04	00	00	00	00	00	00	00	00	00	00	00	00	00

#16 0x0C: **DCR**
A 03->02 B=00 Z=0 PC->0x0D
vars: expo=3 result=4

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	01	41	08	11	01	42	06	40	02	42	01	41	0A	02	41	07
10:	00	FF	00	00	00	00	00	00	00	00	00	00	00	00	00	00
40:	02	03	04	00	00	00	00	00	00	00	00	00	00	00	00	00

#17 0x0D: **STA expo**
A 02->02 B=00 Z=0 PC->0x0F
vars: expo=2 result=4

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	01	41	08	11	01	42	06	40	02	42	01	41	0A	02	41	07
10:	00	FF	00	00	00	00	00	00	00	00	00	00	00	00	00	00
40:	02	02	04	00	00	00	00	00	00	00	00	00	00	00	00	00

#18 0x0F: **JMP loop**
A 02->02 B=00 Z=0 PC->0x00
vars: expo=2 result=4

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	01	41	08	11	01	42	06	40	02	42	01	41	0A	02	41	07
10:	00	FF	00	00	00	00	00	00	00	00	00	00	00	00	00	00
40:	02	02	04	00	00	00	00	00	00	00	00	00	00	00	00	00

#19 0x00: **LDA expo**
A 02->02 B=00 Z=0 PC->0x02
vars: expo=2 result=4

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	01	41	08	11	01	42	06	40	02	42	01	41	0A	02	41	07
10:	00	FF	00	00	00	00	00	00	00	00	00	00	00	00	00	00
40:	02	02	04	00	00	00	00	00	00	00	00	00	00	00	00	00

#20 0x02: **JZ end**
A 02->02 B=00 Z=0 PC->0x04
vars: expo=2 result=4

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	01	41	08	11	01	42	06	40	02	42	01	41	0A	02	41	07
10:	00	FF	00	00	00	00	00	00	00	00	00	00	00	00	00	00
40:	02	02	04	00	00	00	00	00	00	00	00	00	00	00	00	00

#21 0x04: **LDA result**
A 02->04 B=00 Z=0 PC->0x06
vars: expo=2 result=4

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	01	41	08	11	01	42	06	40	02	42	01	41	0A	02	41	07
10:	00	FF	00	00	00	00	00	00	00	00	00	00	00	00	00	00
40:	02	02	04	00	00	00	00	00	00	00	00	00	00	00	00	00

#22 0x06: **MUL base**
A 04->08 B=00 Z=0 PC->0x08
vars: expo=2 result=4

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	01	41	08	11	01	42	06	40	02	42	01	41	0A	02	41	07
10:	00	FF	00	00	00	00	00	00	00	00	00	00	00	00	00	00
40:	02	02	04	00	00	00	00	00	00	00	00	00	00	00	00	00

#23 0x08: **STA result**
A 08->08 B=00 Z=0 PC->0x0A
vars: expo=2 result=8

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	01	41	08	11	01	42	06	40	02	42	01	41	0A	02	41	07
10:	00	FF	00	00	00	00	00	00	00	00	00	00	00	00	00	00
40:	02	02	08	00	00	00	00	00	00	00	00	00	00	00	00	00

#24 0x0A: **LDA expo**
A 08->02 B=00 Z=0 PC->0x0C
vars: expo=2 result=8

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	01	41	08	11	01	42	06	40	02	42	01	41	0A	02	41	07
10:	00	FF	00	00	00	00	00	00	00	00	00	00	00	00	00	00
40:	02	02	08	00	00	00	00	00	00	00	00	00	00	00	00	00

#25 0x0C: **DCR**
A 02->01 B=00 Z=0 PC->0x0D
vars: expo=2 result=8

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	01	41	08	11	01	42	06	40	02	42	01	41	0A	02	41	07
10:	00	FF	00	00	00	00	00	00	00	00	00	00	00	00	00	00
40:	02	02	08	00	00	00	00	00	00	00	00	00	00	00	00	00

#26 0x0D: **STA expo**
A 01->01 B=00 Z=0 PC->0x0F
vars: expo=1 result=8

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	01	41	08	11	01	42	06	40	02	42	01	41	0A	02	41	07
10:	00	FF	00	00	00	00	00	00	00	00	00	00	00	00	00	00
40:	02	01	08	00	00	00	00	00	00	00	00	00	00	00	00	00

#27 0x0F: **JMP loop**
A 01->01 B=00 Z=0 PC->0x00
vars: expo=1 result=8

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	01	41	08	11	01	42	06	40	02	42	01	41	0A	02	41	07
10:	00	FF	00	00	00	00	00	00	00	00	00	00	00	00	00	00
40:	02	01	08	00	00	00	00	00	00	00	00	00	00	00	00	00

#28 0x00: **LDA expo**
A 01->01 B=00 Z=0 PC->0x02
vars: expo=1 result=8

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	01	41	08	11	01	42	06	40	02	42	01	41	0A	02	41	07
10:	00	FF	00	00	00	00	00	00	00	00	00	00	00	00	00	00
40:	02	01	08	00	00	00	00	00	00	00	00	00	00	00	00	00

#29 0x02: **JZ end**
A 01->01 B=00 Z=0 PC->0x04
vars: expo=1 result=8

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	01	41	08	11	01	42	06	40	02	42	01	41	0A	02	41	07
10:	00	FF	00	00	00	00	00	00	00	00	00	00	00	00	00	00
40:	02	01	08	00	00	00	00	00	00	00	00	00	00	00	00	00

#30 0x04: **LDA result**
A 01->08 B=00 Z=0 PC->0x06
vars: expo=1 result=8

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	01	41	08	11	01	42	06	40	02	42	01	41	0A	02	41	07
10:	00	FF	00	00	00	00	00	00	00	00	00	00	00	00	00	00
40:	02	01	08	00	00	00	00	00	00	00	00	00	00	00	00	00

#31 0x06: **MUL base**
A 08->10 B=00 Z=0 PC->0x08
vars: expo=1 result=8

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	01	41	08	11	01	42	06	40	02	42	01	41	0A	02	41	07
10:	00	FF	00	00	00	00	00	00	00	00	00	00	00	00	00	00
40:	02	01	08	00	00	00	00	00	00	00	00	00	00	00	00	00

#32 0x08: **STA result**
A 10->10 B=00 Z=0 PC->0x0A
vars: expo=1 result=16

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	01	41	08	11	01	42	06	40	02	42	01	41	0A	02	41	07
10:	00	FF	00	00	00	00	00	00	00	00	00	00	00	00	00	00
40:	02	01	10	00	00	00	00	00	00	00	00	00	00	00	00	00

#33 0x0A: **LDA expo**
A 10->01 B=00 Z=0 PC->0x0C
vars: expo=1 result=16

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	01	41	08	11	01	42	06	40	02	42	01	41	0A	02	41	07
10:	00	FF	00	00	00	00	00	00	00	00	00	00	00	00	00	00
40:	02	01	10	00	00	00	00	00	00	00	00	00	00	00	00	00

#34 0x0C: **DCR**
A 01->00 B=00 Z=1 PC->0x0D
vars: expo=1 result=16

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	01	41	08	11	01	42	06	40	02	42	01	41	0A	02	41	07
10:	00	FF	00	00	00	00	00	00	00	00	00	00	00	00	00	00
40:	02	01	10	00	00	00	00	00	00	00	00	00	00	00	00	00

#35 0x0D: **STA expo**
A 00->00 B=00 Z=1 PC->0x0F
vars: expo=0 result=16

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	01	41	08	11	01	42	06	40	02	42	01	41	0A	02	41	07
10:	00	FF	00	00	00	00	00	00	00	00	00	00	00	00	00	00
40:	02	00	10	00	00	00	00	00	00	00	00	00	00	00	00	00

#36 0x0F: **JMP loop**
A 00->00 B=00 Z=1 PC->0x00
vars: expo=0 result=16

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	01	41	08	11	01	42	06	40	02	42	01	41	0A	02	41	07
10:	00	FF	00	00	00	00	00	00	00	00	00	00	00	00	00	00
40:	02	00	10	00	00	00	00	00	00	00	00	00	00	00	00	00

#37 0x00: **LDA expo**
A 00->00 B=00 Z=1 PC->0x02
vars: expo=0 result=16

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	01	41	08	11	01	42	06	40	02	42	01	41	0A	02	41	07
10:	00	FF	00	00	00	00	00	00	00	00	00	00	00	00	00	00
40:	02	00	10	00	00	00	00	00	00	00	00	00	00	00	00	00

#38 0x02: **JZ end**
A 00->00 B=00 Z=1 PC->0x11
vars: expo=0 result=16

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	01	41	08	11	01	42	06	40	02	42	01	41	0A	02	41	07
10:	00	FF	00	00	00	00	00	00	00	00	00	00	00	00	00	00
40:	02	00	10	00	00	00	00	00	00	00	00	00	00	00	00	00

#39 0x11: **HLT**
A 00->00 B=00 Z=1 PC->0x12
vars: expo=0 result=16

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
00:	01	41	08	11	01	42	06	40	02	42	01	41	0A	02	41	07
10:	00	FF	00	00	00	00	00	00	00	00	00	00	00	00	00	00
40:	02	00	10	00	00	00	00	00	00	00	00	00	00	00	00	00

Table 14. *Program 2 result*

Result variable	CPU result	Expected (Python)	Same C file via gcc	Instructions executed	Verdict
result @ 0x42	**16** (0x10)	2**4 = 16	16	39	**PASS**

5. How to Run, Files and Team Contributions

Table 15. *Commands (Python 3.8+; only the last one needs a package)*

Command	Effect
`python3 sap8.py`	Print the instruction-encoding table
`python3 prog1_factorial.py`	Program 1: listing + RAM after every instruction
`python3 prog2_power.py`	Program 2: listing + RAM after every instruction
`python3 test_sap8.py`	Self-test (expects: ALL TESTS PASSED)
`gcc prog2_power.c -o power && ./power`	Run the original C program on the host (prints 16)
`pip install reportlab && python3 make_report.py`	Rebuild this PDF from the live code

Table 16. *Project files and owners*

File	Owner	Purpose
`sap8.py`	Sreya (A)	ISA, assembler, CPU simulator, memory-dump printer
`cpu_diagram.svg / .png`	Sreya (A)	Block diagram (Figure 1)
`make_report.py`	Sreya (A)	Generates this PDF
`prog1_factorial.py`	Lavya (B)	5! in assembly
`prog2_power.c`	Manasa (C)	2^4 in C
`prog2_power.py`	Manasa (C)	C translated to SAP-8 assembly
`test_sap8.py`	team	Self-test
`README.md / TASKS.md`	team	Instructions and task division